

BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name:

ID:

Due Date

Thursday, 11/06/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to analytically (or graphically) solve a linear program.
- Problem 2 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 3 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

Activating project at `~/Documents/GitHub/hw4-sarah-and-nathan-1`

```

using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables

```

Problems (Total: 30 Points)

Problem 1 (Total: 3 Points)

Solve the following linear program **analytically**. Show your work and justify any reasoning.

$$\begin{aligned}
 & \max_{x,y} && 3x + 2y \\
 & \text{subject to:} && x + 4y \leq 16 \\
 & && x - 2y \leq -1 \\
 & && 0 \leq x \leq 4 \\
 & && 1 \leq y \leq 4.
 \end{aligned}$$

Problem 2 (12 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs \$70/kg-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem 1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost (\$/ha-yr)	Selling Price (\$/kg)
Soybeans	350	0.36
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

Problem 2.1

Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear.**

Problem 2.2

How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?

Problem 2.3

The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and under what conditions you would recommend the farmer should make the purchase.

Problem 2 (15 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

```

#2.1
using JuMP
using HiGHS

#Data from question
farm_model = Model(HiGHS.Optimizer) #initializing farm model
crops = [:soy, :wheat, :corn] #initializing different crop types
Pest_rates = [0, 1, 2] #Application rates
Production_cost = Dict(:soy=>350, :wheat=>280, :corn=>390) #production cost of each crop $/ha
price = Dict(:soy=>0.36, :wheat=>0.27, :corn=>0.22) #Selling price $/kg
Pestrate_limit= Dict(:soy=>0.8, :wheat=>0.7, :corn=>0.6) #regulatory average limits (kg/ha)
pesticide_cost = 70 # $/kg
land_total = 130 #ha
demand = 250000 #kg/crop

#yields for each crop at each pesticide application rate (kg/ha)
yield_table = Dict(
    :soy => Dict(0=>2900, 1=>3800, 2=>4400),
    :wheat => Dict(0=>3500, 1=>4100, 2=>4200),
    :corn => Dict(0=>5900, 1=>6700, 2=>7900)
)

#area variables: A[i,r] = hectares of crop i planted at pesticide rate r
@variable(farm_model, A[i in crops, r in Pest_rates] >= 0)

#maximize profit = revenue - base costs - pesticide cost
@objective(farm_model, Max,
    sum( A[i,r] * ( price[i] * yield_table[i][r] - base_cost[i] - pesticide_cost * r )
        for i in crops, r in Pest_rates)
)

#land constraint
@constraint(farm_model, sum(A[i,r] for i in crops, r in Pest_rates) <= land_total)

#Demand cap per crop (total production of crop 1,2,3 <= demand)
@constraint(farm_model, [i in crops], sum(A[i,r] * yield_table[i][r] for r in Pest_rates) <= demand)

#Regulatory application rate per crop:
@constraint(farm_model, [i in crops],
    sum(r * A[i,r] for r in Pest_rates) <= Pestrate_limit[i] * sum(A[i,r] for r in Pest_rates)
)

```

```

#Solve
optimize!(farm_model)

#2.2

#Print the results of each crop
println("Objective (profit) = \$", objective_value(farm_model))
for i in crops
    println("Crop: ", i)
    for r in rates
        ha = value(A[i,r])
        if ha > 1
            println("rate = ", r, "kg/ha, area = ", ha, "ha, production = ", ha * yield_table[i,r])
        end
    end
end

total_profit = objective_value(farm_model)
println("Total Profit: \$", total_profit)

```

Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms

LP has 7 rows; 9 cols; 27 nonzeros

Coefficient ranges:

Matrix [2e-01, 8e+03]

Cost [7e+02, 1e+03]

Bound [0e+00, 0e+00]

RHS [1e+02, 2e+05]

Presolving model

7 rows, 9 cols, 27 nonzeros 0s

7 rows, 9 cols, 27 nonzeros 0s

Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced

Problem not reduced by presolve: solving the LP

Using EKK dual simplex solver - serial

Iteration	Objective	Infeasibilities	num(sum)
0	-1.8170298458e+02	Ph1: 7(28.3601); Du: 9(181.703)	0s
7	1.1674116702e+05	Pr: 0(0)	0s

Model status : Optimal

Simplex iterations: 7

Objective value : 1.1674116702e+05

P-D objective error : 0.0000000000e+00

HiGHS run time : 0.00

Objective (profit) = \$116741.16702082448

```

Crop: soy
rate = 0kg/ha, area = 13.812154696132604ha, production = 40055.24861878455kg, profit = $9585
rate = 1kg/ha, area = 55.24861878453038ha, production = 209944.75138121544kg, profit = $5237
Crop: wheat
rate = 0kg/ha, area = 6.743306417339563ha, production = 23601.572460688472kg, profit = $4484
rate = 1kg/ha, area = 15.734381640458977ha, production = 64510.96472588181kg, profit = $1191
Crop: corn
rate = 0kg/ha, area = 26.92307692307692ha, production = 158846.15384615381kg, profit = $2444
rate = 2kg/ha, area = 11.538461538461547ha, production = 91153.84615384623kg, profit = $1393
Total Profit: $116741.16702082448

```

```

#2.3
using JuMP
using HiGHS

#Data from question
farm_model = Model(HiGHS.Optimizer) #initializing farm model
crops = [:soy, :wheat, :corn] #initializing different crop types
Pest_rates = [0, 1, 2] #Application rates
Production_cost = Dict{:soy=>350, :wheat=>280, :corn=>390} #production cost of each crop $/ha
price = Dict{:soy=>0.36, :wheat=>0.27, :corn=>0.22} #Selling price $/kg
Pestrate_limit= Dict{:soy=>0.8, :wheat=>0.7, :corn=>0.6} #regulatory average limits (kg/ha)
pesticide_cost = 70 # $/kg
land_total = 140 #ha
demand = 250000 #kg/crop

#yields for each crop at each pesticide application rate (kg/ha)
yield_table = Dict(
  :soy => Dict(0=>2900, 1=>3800, 2=>4400),
  :wheat => Dict(0=>3500, 1=>4100, 2=>4200),
  :corn => Dict(0=>5900, 1=>6700, 2=>7900)
)

#area variables: A[i,r] = hectares of crop i planted at pesticide rate r
@variable(farm_model, A[i in crops, r in Pest_rates] >= 0)

#maximize profit = revenue - base costs - pesticide cost
@objective(farm_model, Max,
  sum( A[i,r] * ( price[i] * yield_table[i][r] - base_cost[i] - pesticide_cost * r )
    for i in crops, r in Pest_rates)
)

```

```

#land constraint
@constraint(farm_model, sum(A[i,r] for i in crops, r in Pest_rates) <= land_total)

#Demand cap per crop (total production of crop 1,2,3 <= demand)
@constraint(farm_model, [i in crops], sum(A[i,r] * yield_table[i][r] for r in Pest_rates) <= demand[i])

# Regulatory application rate per crop:
@constraint(farm_model, [i in crops],
    sum(r * A[i,r] for r in Pest_rates) <= Pestrates_limit[i] * sum(A[i,r] for r in Pest_rates)
)

#Solve
optimize!(farm_model)

#Print the results of each crop
println("Objective (profit) = \$", objective_value(farm_model))
for i in crops
    println("Crop: ", i)
    for r in rates
        ha = value(A[i,r])
        if ha > 1
            println("rate = ", r, "kg/ha, area = ", ha, "ha, production = ", ha * yield_table[i][r])
        end
    end
end
end

total_profit = objective_value(farm_model)
println("Total Profit: \$", total_profit)

```

```

Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP      has 7 rows; 9 cols; 27 nonzeros
Coefficient ranges:
  Matrix [2e-01, 8e+03]
  Cost    [7e+02, 1e+03]
  Bound   [0e+00, 0e+00]
  RHS     [1e+02, 2e+05]
Presolving model
7 rows, 9 cols, 27 nonzeros  0s
7 rows, 9 cols, 27 nonzeros  0s
Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced
Problem not reduced by presolve: solving the LP
Using EKK dual simplex solver - serial

```

```

Iteration      Objective      Infeasibilities num(sum)
      0    -1.8170298458e+02 Ph1: 7(28.3601); Du: 9(181.703) 0s
      7      1.2403516702e+05 Pr: 0(0) 0s
Model status      : Optimal
Simplex iterations: 7
Objective value    : 1.2403516702e+05
P-D objective error : 5.8660204673e-17
HiGHS run time     : 0.00
Objective (profit) = $124035.16702082449
Crop: soy
rate = 0kg/ha, area = 13.8121546961326ha, production = 40055.24861878454kg, profit = $9585.6
rate = 1kg/ha, area = 55.24861878453038ha, production = 209944.75138121544kg, profit = $5237
Crop: wheat
rate = 0kg/ha, area = 9.743306417339566ha, production = 34101.57246068848kg, profit = $6479.
rate = 1kg/ha, area = 22.734381640458977ha, production = 93210.9647258818kg, profit = $17209
Crop: corn
rate = 0kg/ha, area = 26.923076923076938ha, production = 158846.15384615393kg, profit = $244
rate = 2kg/ha, area = 11.538461538461544ha, production = 91153.8461538462kg, profit = $13938
Total Profit: $124035.16702082449

```

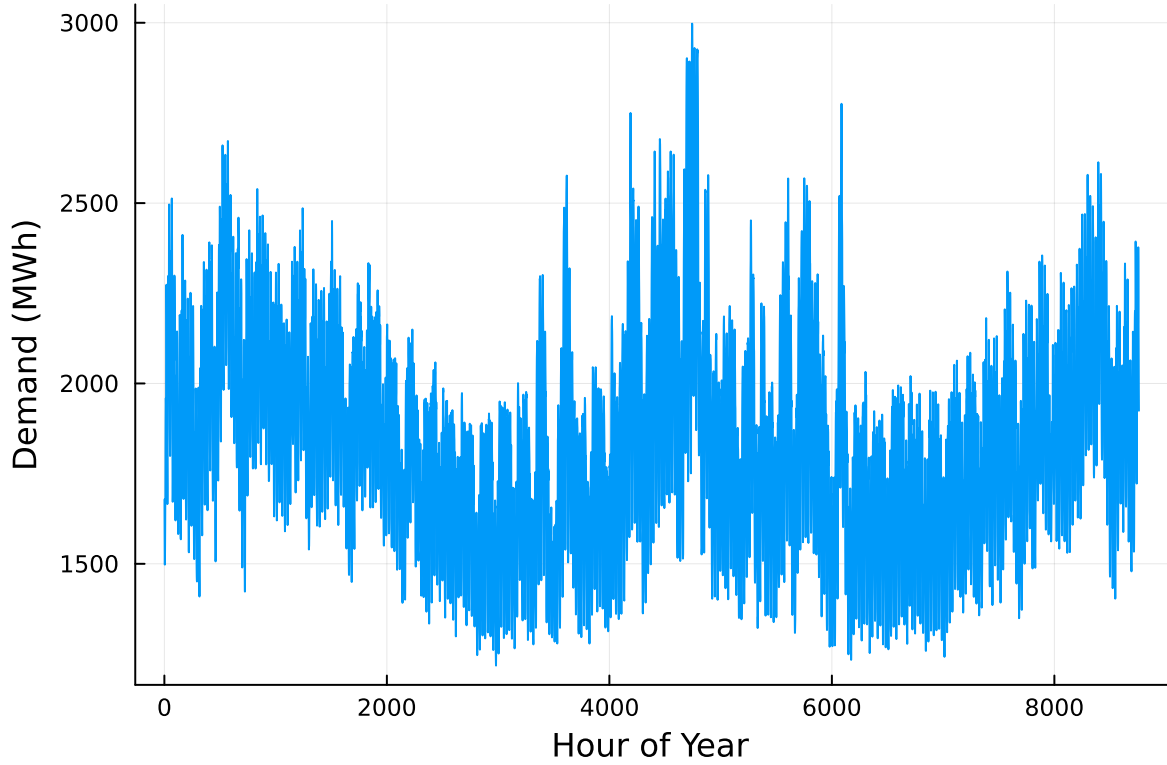
2.3) The farmer should expect to make around \$7249 more dollars, adding another 10 ha of land. This increased profit comes from the ability to grow more wheat, as soy and corn are already maxed out in production. While we cannot sell any more corn or soy, we can sell more wheat. The extra 10ha does not maximize the wheat able to be sold, so this extra profit would make sense under certain conditions. These conditions include: if there is sufficient equipment to grow and harvest the crops, or if there is enough space in the trucks to move the wheat to a selling location.

```

# load the data, pull Zone C, and reformat the DataFrame
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, : "Time Stamp" => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)

# plot demand
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:false)

```

Next, we load the generator data, shown in Table 3. This data includes fixed costs (\$/MW installed), variable costs (\$/MWh generated), and CO2 emissions intensity (tCO2/MWh generated).

```
gens = DataFrame(CSV.File("data/generators.csv"))
```

	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

```

# load capacity factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
axis!("Hour of the Month")
axis!("Capacity Factor")

p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
axis!("Hour of the Month")
axis!("Capacity Factor")

display(p1)
display(p2)

```

You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind. The NY state legislature is planning to enact an annual CO₂ limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO₂/yr.

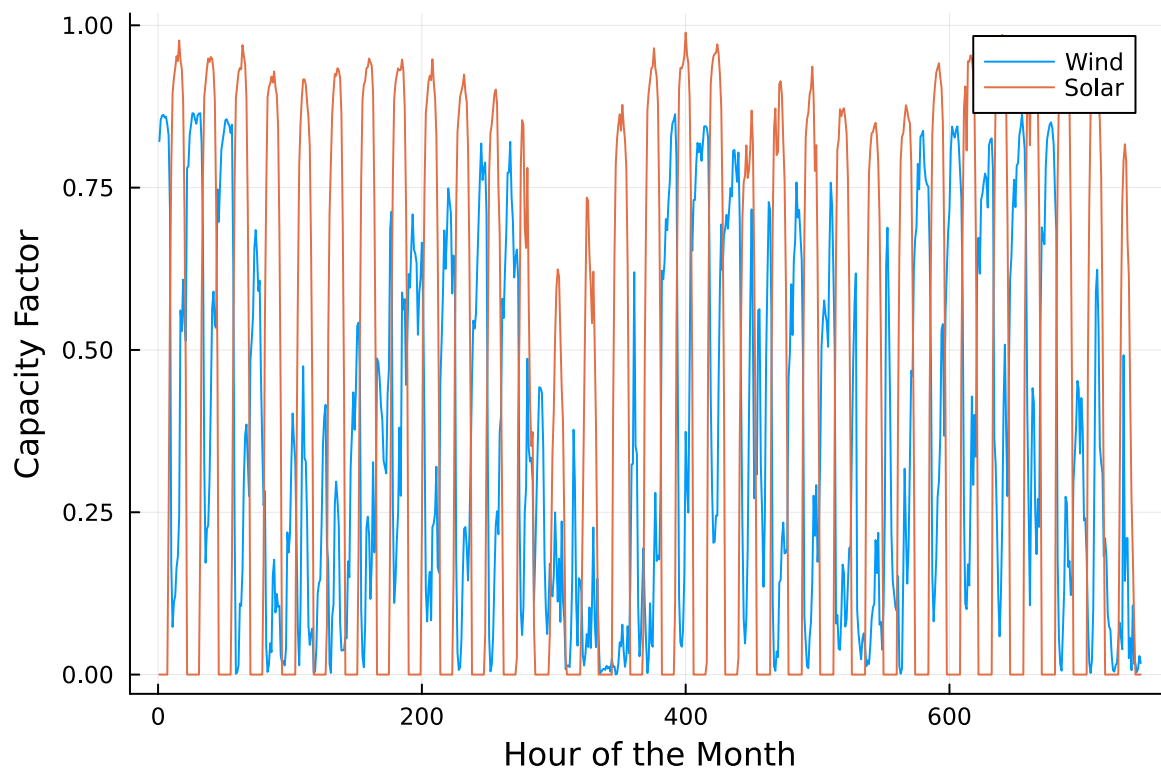
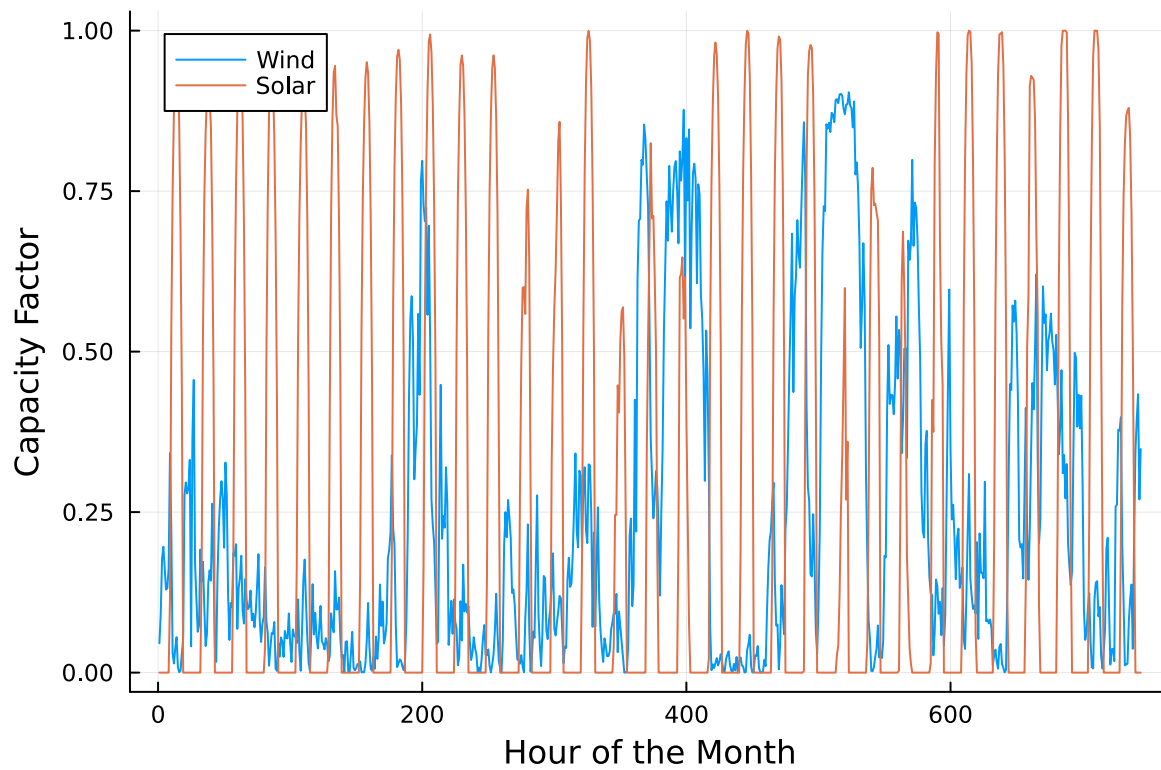
While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of \$10,000/MWh.

Problem 3.1

Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).

Problem 3.2

How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?



Problem 3.3

What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 3.2? Do these results make sense given the demand and generator data?

Problem 3.4

Make a plot of the electricity price in each hour. Discuss any trends that you see.

Problem 3.5

What is the shadow price of CO₂? What is the value to the utility be of allowing it to emit an additional 1000 tCO₂/yr?

Significant Digits

Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.

Getting Variable Output Values

`value.(x)` will report the values of a JuMP variable `x`, but it will return a special container which holds other information about `x` that is useful for JuMP. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.

Suppressing Model Command Output

The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing output by adding a semi-colon after the last command, or you might find that your notebook crashes.

References

List any external references consulted, including classmates.

Stack overflow for dict help